

Top tip!

Spend significant time on data preprocessing, visualization and analysis.

Understanding the data is as important as understanding the algorithms.

◆ **Outlier:** a data point that deviates from the typical pattern of values in a data set, indicating a possible unusual or erroneous value that should be discounted.

SYLLABUS CONTENT

By the end of this chapter, you should be able to:

- ▶ A4.2.1 Describe the significance of data cleaning
- ▶ A4.2.2 Describe the role of feature selection
- ▶ A4.2.3 Describe the importance of dimensionality reduction

A4.2.1 Data cleaning

High-quality data builds high-quality models. If the training data is full of errors or redundant features, the model will learn from these inaccuracies and make poor predictions.

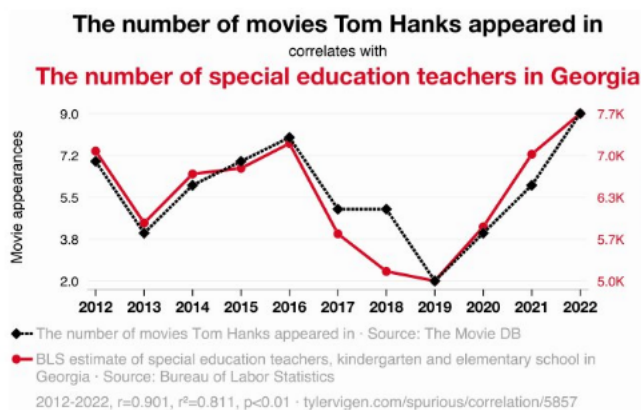
Taking the time to ensure your data is as clean as possible will reap rewards with respect to efficiency and accuracy. There are several steps that may be useful for cleaning your data set.

- 1 **Handling outliers:** Statistical methods, such as using the interquartile range or Z-scores, can detect outlying data. Once found, depending on the context, outlying data may be capped, transformed or removed as appropriate.

Python

```
import numpy as np
# Create random array of values between 0 and 100
# Set one extreme value to act as an outlier
data = np.random.randint(0, 100, size=1000)
data[999] = 937
# Calculate outliers via Z-scores
mea = np.mean(data)
std_dev = np.std(data)
z_scores = (data - mea) / std_dev
threshold = 3 # Outliers if 3 stddev from mean
outliers = data[np.abs(z_scores) > threshold]
print("mean", mea, "stddev", std_dev)
print("Outliers:", outliers)
# Calculate outliers via IQR
q1 = np.percentile(data, 25)
q3 = np.percentile(data, 75)
iqr = q3 - q1
cutoff = 1.5 * iqr
lower_bound = q1 - cutoff
upper_bound = q3 + cutoff
outliers = data[(data < lower_bound) | (data > upper_bound)]
print("Outliers:", outliers)
```

- 2 **Removing duplicate data:** Identifying and removing duplicate data will assist in preventing the model from becoming biased towards over-represented values. For data sets where individual records contain a large number of variables, calculating and comparing SHA256 hash values can be a useful mechanism for detecting duplicates (see Section B4.1.6 for more about hash values). Depending on the context of the model, near-duplicate data may also need to be consolidated into a single record.
- 3 **Identifying incorrect data:** Process your data through validation rules to ensure obviously incorrect data can be found and removed. This may mean checking the ranges given for dates and times, or amounts given for currency values, and so on. Set sensible limits and have your program detect anomalies for possible manual checking.
- 4 **Filtering irrelevant data:** If there is no measurable correlation between an input variable and the outcome variable, it may be completely irrelevant and contribute nothing to the predictive power of the model. Keeping such data in the training process is only going to make the process less efficient and less accurate. Additionally, just because data may appear to be correlated doesn't mean it is. As the Spurious Correlations website demonstrates, if you compare enough unrelated data sets, you will find correlations that are, in fact, not.



■ Tom Hanks movies vs special education teachers in Georgia



■ Robberies in Alaska vs professor salaries

- 5 **Transform improperly formatted data:** Data may be incorrectly formatted but easily correctable to ensure consistency in what is presented to the machine learning model, for example:
 - ☐ Ensure all dates are in a consistent style (not having a mix of day / month / year, month / day / year, or ISO yyyy-mm-dd formats).
 - ☐ Ensure numerical values are formatted, and to the same level of precision.
 - ☐ Ensure images are correctly rotated and oriented, and of matching ratio and size.
- 6 **Missing data:** Sometimes it may be necessary to use models to predict missing values to ensure full coverage of the data set. Mean / mode imputation, k-nearest neighbours or regression models could be used for this, if required.
- 7 **Normalization and standardization:** Many machine learning algorithms will benefit from completing preprocessing of data by performing the statistical operations of normalization and standardization to scale data to a standard range or distribution.
 - ☐ Normalization can be used to rescale input data to a range of [0,1] or [-1,1], which is useful when various features (input variables) have different scales.
 - ☐ Standardization can be used to transform the input data to have a mean score of 0 and standard deviation of 1 (Gaussian distribution). (Note that it is not mathematically possible for the range to be [-1,1] and to have a standard deviation of 1; you need to determine which is required for your model.)

Common mistake

Ignoring the important role that normalization and standardization play

Recognize that normalization (scaling data to a range) and standardization (scaling data to have zero mean and unit variance) are crucial for many algorithms to perform optimally. Apply these transformations consistently across all data used in the model.

◆ **Feature:** a numeric property that can be used to contribute a data point for a machine learning algorithm to train on. Think of it as a variable in your data set.

Common mistake

Don't underestimate the importance of feature selection and engineering. Good features are often more important than the choice of model itself.

Python

```
import numpy as np
data = np.array([10, 20, 30, 40, 50])
# Normalize the data to have a mean of 0, and have range [-1, 1]
data_mean_centered = data - np.mean(data)
max_abs_val = np.max(np.abs(data_mean_centered))
normalized_data = data_mean_centered/max_abs_val
print(normalized_data)
# Standardize the data to have a mean of 0, and std dev of 1
standardized_data = (data - np.mean(data))/np.std(data)
print(standardized_data)
```

A4.2.2 Feature selection

TOK

How does the way that we organize or classify knowledge affect what we know?

The structuring of data sets and the choice of features directly influence the insights gained from machine learning algorithms.

The way data is structured can significantly determine what the machine learning model can learn. For instance, missing values; the inclusion or exclusion of certain data points; or the way categories are defined and labelled can all skew or bias the model's outputs. This structuring determines how the machine "views" and "understands" the world, directly influencing the patterns it recognizes and the predictions it makes.

The features chosen can amplify or suppress certain patterns within the data. For example, in a model predicting creditworthiness, choosing features like income might reflect economic factors, whereas including features like zip code could inadvertently introduce socio-economic biases related to geographical areas.

The decisions made in data structuring and feature selection are not value-neutral. They reflect the biases, perspectives and priorities of those who design the data sets and algorithms.

Feature selection refers to taking care to select only the most relevant features for use in your machine learning models. In the context of machine learning, a **feature** is a variable that you wish to use as input values for generating predictions. While it may seem like a lot of additional effort to perform manual feature selection, the process can dramatically impact the overall performance and accuracy of your machine learning model.

Removal of irrelevant detail will result in a more generalized model that is better suited to processing new, previously unseen data.

Three commonly used methods to help determine which features to select are filter methods, wrapper methods and embedded methods.

See also

For more detail on these approaches, along with example code, search online for scikit-learn's section 1.13 "Feature selection" documentation (https://scikit-learn.org/stable/modules/feature_selection.html).

Filter methods

So-called as they help “filter out” features, filter methods involve applying a statistical metric to determine which features are best to be retained and which should be removed from the model. Features are ranked by their score, and those that don’t meet the threshold can be filtered out.

As a purely statistical measure, using filter methods is less computationally expensive than retaining the feature in the model for full training. The downside is this does not detect interaction between features. That is, if one feature is affecting another, then a filter may suggest deleting a feature that is actually important. This is where manual appreciation of the context of your model is always important.

The most common, and easy-to-use, filter is to calculate the r value of the correlation (Pearson’s product moment correlation coefficient). The r value of a data set may be calculated using

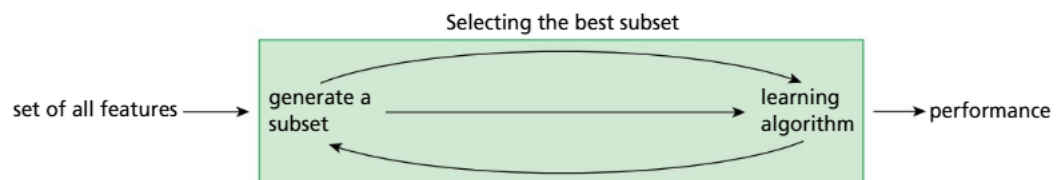
$$r = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum(x_i - \bar{x})^2 \sum(y_i - \bar{y})^2}}$$

where x_i and y_i are your individual data points and $\bar{x}$ and $\bar{y}$ are the mean of each data series.

Once calculated, records with r values beyond a given threshold can be flagged for deletion.

Wrapper methods

Wrapper methods involve iterating over different combinations of the input features and comparing which subset produces optimal performance.



Wrapper methods

This can be a time-consuming and computationally expensive process, especially when compared to filter methods. There is also an increased risk of overfitting the model. The benefit, however, can be a very quick and efficient final model at the end of the process.

For further study on suitable techniques, do some research into recursive feature elimination (RFE), and sequential feature selection (forward selection, backward elimination). The scikit-learn library (online) provides functionality for both.

Embedded methods

Embedded methods draw on both filter and wrapper methods, but incorporate them directly into the model training algorithm. This means that the feature selection is performed simultaneously with the model training, rather than as a separate step before training.

Embedded methods can be more computationally efficient since they don’t require separate iteration of the data prior to training. An embedded method will automatically assess the relevance or importance of features and adjust their weights or inclusion in the model accordingly during the training process.

While embedded methods can save manual labour by eliminating the need for feature selection processes prior to training, they typically require more computational time compared to simpler filter methods. The effectiveness of embedded methods depends on the model’s ability to accurately assess feature relevance during the training process.

ACTIVITY

Research skills:

Select and analyse an existing open-source data set relevant to a specific machine learning problem. Learn about the data cleaning and feature selection process used by these “professional” projects, and make recommendations for students learning to use data-cleaning methods for the first time.

A4.2.3 Dimensionality reduction

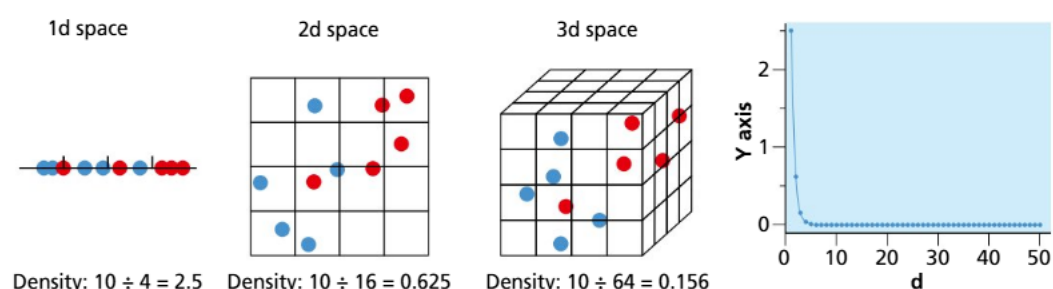
When getting started with machine learning, it is easy to make the mistake of giving too much data to your model. While more quality entries in your data set is usually good, supplying too many features for each entry can easily cause more harm than good.

A typical way of thinking about this as a beginner would be “The more attributes or features I supply, the more detail about my data the model will learn, and perhaps it’ll discover a pattern that I hadn’t thought of”. The problem is that machine learning algorithms are at their best when they are able to make generalizations about the training data. If there is too much detail in each item, and not enough items overall to compensate for that extra detail, then challenges arise.

These challenges are known as the **curse of dimensionality**, and describe the problems that arise in highly dimensional data. The following visualization is a useful way to help understand the problem.

◆ **Curse of dimensionality:** each feature in a machine learning model adds another dimension to the overall model the algorithm is attempting to map and create generalizations about; the curse of dimensionality refers to the problem that occurs when there are too many dimensions relative to the quantity of data available, so that patterns cannot be meaningfully observed.

◆ **Data sparsity:** how “spread out” data points are from each other in a model.



■ Data increases in sparsity as more dimensions are added

In the first panel, there are 10 data points in one dimension, which represents one feature or variable that the model is training with. With 10 points spread across a range of [0,4], there are 2.5 data points per unit. Visually, you can see it is quite crowded, meaning there is a lot of data available to make conclusions and generalizations from.

In the second panel, the same 10 data points are now spread across two dimensions. While both dimensions still have the range [0,4], the effect of the extra dimension is that it squares the space available, so those 10 data points now spread out such that there are only 0.625 data points per unit.

In the third panel, the third dimension is added. With three dimensions, representing three features or variables, there is now only one data point per 0.156 units of space.

The additional detail that comes from adding the extra dimensions acts to spread the data out, making it a lot more difficult for the model to find the generalizations it needs to be useful. To keep the ratio of data points to space consistent, the third panel needs 160 items in its training data instead of just 10. If you don’t compensate for additional dimensions with additional quantity of data, the quality of your model will deteriorate.

The empty cells in the diagram above are an example of **data sparsity**, which is where the data points are too far from each other, and the data set contains a high number of empty values. If asked to generate a prediction when given those values that are empty in the training set, the model will have no basis on which to make an accurate estimation.

Sparsity is problematic as it makes it difficult for models to find patterns without overfitting, which is where the model effectively memorizes the individual items in the data set, including the noisy little details.

Distance metrics, such as calculating the Euclidean distance between points, lose meaning as the distances between all pairs of points are similar (a long way away). Without being able to find the patterns needed to make generalizations, the model will not be useful with unseen, untrained data when you need it to be.

An over-abundance of dimensions also poses challenges for you and those on your developer team with respect to data visualization. The mental capacity required to visualize highly dimensional data is very difficult or impossible, and most humans struggle to wrap their mind around more than three dimensions. This lack of intuition will make it difficult to analyse the patterns and relationships within your data.

While increasing the sample size will help compensate for additional dimensions with respect to model accuracy, it does introduce its own issues. Large sample sizes require increased processing time and capacity for training, and increase memory usage requirements. Every additional dimension already adds an extra order of magnitude to the memory required by the model, so the increase in sample size required not to lose model accuracy only exacerbates the memory and processing requirements. This all works to reduce the accessibility of your model for limited hardware environments such as mobile or portable computing, home / office computing and those without specialized infrastructure such as GPUs.

For these reasons, it can often be better to reduce rather than increase the number of dimensions in your machine learning model. Keeping it simple will help the model learn the generalizations it needs and reduce the demands on your limited processing hardware.



Common mistake

Misunderstanding the goals of dimensionality reduction

It is important to bear in mind that dimensionality reduction does not *always* lead to better model performance. The primary goal of dimensionality reduction is to simplify the model by reducing the number of variables, which can help in some cases but might also lead to loss of critical information. There is a careful balance to be struck with the retention of relevant data aspects, which can take a lot of practice to get right.

■ Reducing dimensions of existing data sets

You can either make decisions about which dimensions to reduce manually, or make use of statistical tools to assist in the process.

Two commonly used statistical techniques to help reduce the number of features are PCA (principal component analysis) and LDA (linear discriminant analysis). PCA and LDA are beyond the scope of your course, but you don't have to know how they work to be able to make use of them in your IA (if you wish or need to), as scikit has the functionality built in.

PCA is used for dimensionality reduction without considering your data set labels. It's good for data compression, visualization and speeding up learning algorithms by reducing the number of input variables.

LDA is supervised in the sense that it uses your training data labels. It reduces a data set to a specified number of dimensions in a manner that best discriminates between the classes, based on their statistical properties. For this reason, it is particularly used to prepare data for classification tasks.

Top tip!

Sklearn, also known as scikit-learn, is a Python module for machine learning built on top of SciPy and distributed under the 4-Clause BSD licence.

To install this module, please refer to the instructions at <https://scikit-learn.org/stable/install.html>

Python

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import
LinearDiscriminantAnalysis as LDA
# Generate synthetic data
# x will be 2d array of 1000 rows, 20 columns
# y will be 1d array of integers of values 0, 1 or 2
x, y = make_classification(n_samples=1000, n_features=20,
                           n_informative=10, n_redundant=10,
                           n_clusters_per_class=1, n_classes=3)
print("Original Data Shape:", x.shape)
# PCA
# Transform sample data from 1000x20 to 1000x2
pca = PCA(n_components=2)
x_reduced_pca = pca.fit_transform(x)
print("PCA Reduced Data Shape:", x_reduced_pca.shape)
# LDA
# Transform sample data from 1000x20 to 1000x2
lda = LDA(n_components=2)
x_reduced_lda = lda.fit_transform(x, y)
print("LDA Reduced Data Shape:", x_reduced_lda.shape)
```

See also

For more about using scikit-learn tools to help reduce dimensions in your data sets, search online for scikit-learn's section 1.2 "Linear and Quadratic Discriminant Analysis" (https://scikit-learn.org/stable/modules/lda_qda.html).

Top tip!

For discussing data preprocessing needs in an examination setting, be sure to attain a thorough understanding of the different role and effect of each one.

Some key points to remember for each one:

- **Inputting missing data:** The input of missing data will improve model accuracy through providing a complete data set, but it can introduce bias if the resulting data set does not match actual data distribution.
- **Deleting missing data:** Simplifies the model by removing incomplete cases to reduce overfitting, but this could lead to the loss of valuable data.
- **Removing duplicates:** Enhances reliability and prevents skewing of results.
- **Removing outliers:** The model can become more generalized as it prevents extreme values from disproportionately influencing predictions.
- **Filtering irrelevant features:** By concentrating on what is most relevant, the model will perform better and faster. It reduces the risk of overfitting.
- **Normalization and standardization:** Most algorithms perform better when features are all on a similar scale.
- **Filter methods:** Computationally less expensive, they can be used regardless of model type.
- **Wrapper methods:** Provide better performance as they consider feature interaction and are tailored to the model in question. If the data set is small, it can lead to overfitting.
- **Embedded methods:** A balanced approach between filter and wrapper methods, their effectiveness is dependent on the model they are designed for.

REVIEW QUESTIONS

- 1 A marketing firm uses machine learning to analyse customer survey data to improve targeting strategies.
 - a Describe one common issue in survey data that would necessitate data cleaning.
 - b Describe how feature selection could impact the performance of a machine learning model in this scenario.
 - c Outline the role of dimensionality reduction in handling high-dimensional data such as survey responses.
- 2 A financial analytics firm uses machine learning to predict stock-market trends based on historical data.
 - a List one common data-quality issue that might require cleaning in this historical stock data.
 - b Describe the role of feature selection in improving model performance in financial predictions.
 - c Describe the importance of dimensionality reduction on model complexity and performance.
- 3 A school district analyses standardized test results to predict student performance and identify at-risk students.
 - a List one common data issue that might arise with standardized test-result data.
 - b Describe the possible implications if the school district was to import raw test data for all questions completed by students into the machine learning model.
 - c Outline two commonly used methods of feature selection that could be beneficial in this educational context.